

Nano-Kernel Runtime (NKR): A Bare-Metal Micro-VM Hypervisor for Multi-Tenant SaaS Workloads

White Paper — Version 1.0

April 2026

Contents

1	Introduction and Motivation	4
1.1	The Problem	4
1.2	What is NKR?	4
2	Design Goals	5
3	Architectural Overview	5
4	VMM Engine: From KVM to Boot	6
4.1	KVM Initialization	6
4.2	Guest Memory Map (x86_64)	7
4.3	Boot Protocol	7
4.4	Time and Clock Management (Clock Synchronization)	8
4.5	vCPU Loop	8
5	VirtIO Device Model	8
5.1	MMIO Address Map	9
5.2	VirtIO Block Device	9
5.3	VirtIO Network Device	9
6	Networking	10
6.1	Bridge Topology	10
6.2	Automatic Configuration	10
6.3	Port Forwarding	10
7	Disk Lifecycle: From OCI to ext4	11
7.1	Build Pipeline	11
7.2	Nkrfile Format	11
7.3	Copy-on-Write Snapshots	12
7.4	Volumes	12
7.5	Environment Variables	12

8 The CPU Model: “Chrs”	12
8.1 Implementation	13
9 Initramfs Generation	13
9.1 Boot Sequence	13
9.2 Automatic Entrypoint Detection	14
10Orchestration with Compose	15
10.1Compose File Format	15
10.2Resource Resolution	15
10.3Features	16
10.4NKR Data Directory	16
11Multi-Tenant Deployment	16
11.1Tenant Registry	17
11.2Provisioning Pipeline	17
11.3Hot Module Update	17
11.4Target Architecture	18
12Observability and Metrics	19
13Comparison with Existing Solutions	19
13.1NKR vs Docker	19
13.2NKR vs Firecracker	20
13.3NKR vs QEMU/KVM	21
14Security Model	21
14.1Isolation Boundaries	21
14.2Attack Surface	22
14.3Operational Security	22
15Limitations and Future Work	22
15.1Current Limitations	22
15.2Roadmap	23
16Conclusion	24
17Appendix A: Technology Stack	25
18Appendix B: Source Code Metrics	25
19Appendix C: Quick Start	26

Abstract. The Nano-Kernel Runtime (NKR) is an open-source bare-metal hypervisor written in Rust that replaces container runtimes like Docker with hardware-isolated micro-VMs, running directly on Linux KVM. NKR is designed for operators managing dense multi-tenant SaaS deployments—especially Odoo ERP—on a single server with limited resources (16-32 GB RAM). By eliminating the overhead of QEMU, libvirt, and container-level sharing, NKR achieves full hardware isolation with a binary of just 2-4 MB, VM boot times under one second, exclusive CPU scheduling (the “chrs” model), and a Docker-compatible workflow for building disk images. This document presents the architecture, implementation, and production deployment model of NKR.

1 Introduction and Motivation

1.1 The Problem

Service providers managing dozens of SaaS tenants on shared infrastructure face a fundamental tension between **density** (maximizing the number of tenants per server) and **isolation** (avoiding the noisy neighbor effect). Docker containers offer high density but share the host kernel, exposing a large attack surface and lacking strict CPU or RAM guarantees. Traditional VMs (QEMU/KVM with libvirt) provide solid isolation but impose prohibitive memory and disk overhead for dense deployments.

Consider a practical scenario: an operator managing **50 Odoo 17 ERP instances** on a single 16–32 GB server using Docker:

Issue	Impact with Docker	Impact with NKR
Disk Usage	50 × 1.5 GB images ≈ 75 GB	Shared ext4 base + CoW snapshots
RAM Consumption	50 × ~1 GB ≈ 50 GB	50 × ~256 MB ≈ 12.5 GB (exclusive)
CPU Contention	Shared scheduler, no guarantees	Pinned cores with the “chrs” model
Restart Latency	~3 minutes per stack restart	~5 seconds (hot updates)
Deployment Cycle	git pull → rebuild → restart	git pull → rsync → restart Odoo only
Infrastructure Footprint	50 Odoo + 50 PostgreSQL + 50 nginx	N Odoo + 1 PostgreSQL + 1 nginx

NKR was created to eliminate these compromises, providing VM-level isolation with the operational simplicity of a container.

1.2 What is NKR?

Nano-Kernel Runtime (NKR) is a purpose-built hypervisor that:

- Runs micro-VMs directly over `/dev/kvm` without QEMU, libvirt, or containerd
- Provides each “container” with a real Linux kernel, an ext4 root filesystem, and VirtIO devices

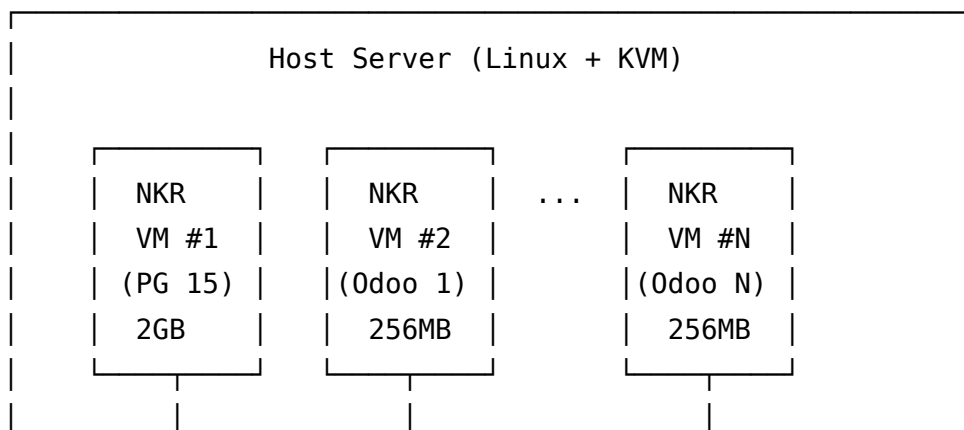
- Compiles to a **single ~2-4 MB binary** (Rust, LTO, stripped)
- Offers a Docker-compatible CLI (`nkr run`, `nkr ps`, `nkr stop`, `nkr compose up`)
- Uses Docker **only** at build time to generate disk images from OCI/Dockerfiles

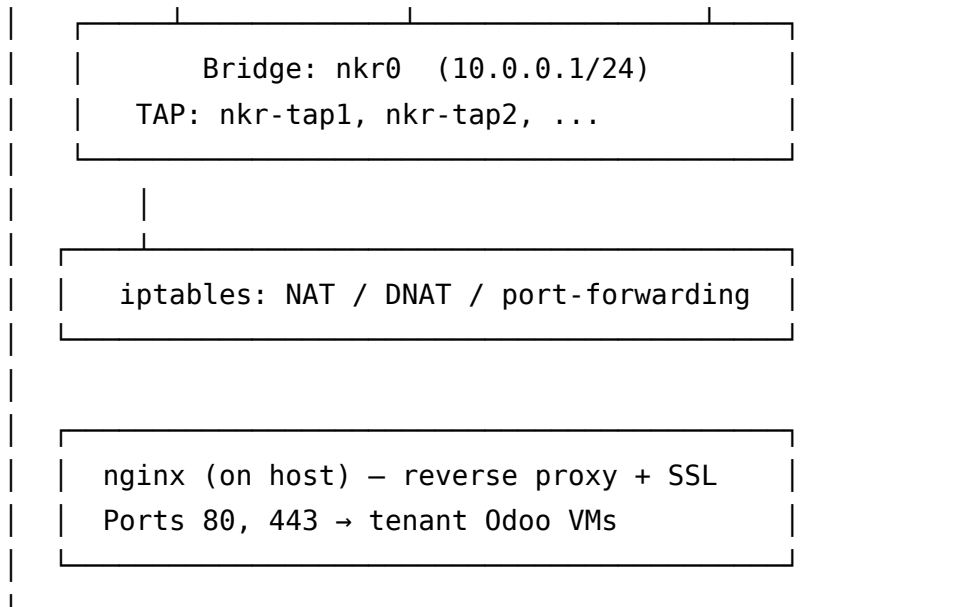
2 Design Goals

The design of NKR is guided by five principles:

1. **Zero runtime external dependencies.** The `nkr` binary only requires a Linux kernel with KVM support. No QEMU, no libvirt, no container runtime.
2. **Hardware isolation with container ergonomics.** Each workload runs in a full KVM virtual machine—with its own kernel, page tables, and interrupt controller—even though operators interact with it using familiar Docker-style commands and compose files.
3. **Deterministic resource allocation.** RAM is mapped exclusively to each VM. CPU cycles are guaranteed via core pinning. There is no memory overcommit.
4. **Minimal footprint.** The hypervisor binary weighs 2-4 MB. Guest overhead is bounded: a 256 MB VM uses exactly 256 MB of host RAM.
5. **Production-ready for multi-tenant SaaS.** First-class support for multi-tenant Odoo deployments with shared PostgreSQL, hot module updates, and automated provisioning.

3 Architectural Overview





Each micro-VM is a complete virtual machine featuring:

- A Linux kernel (bzImage, shared binary among all VMs)
- An ext4 root filesystem (created from OCI images)
- VirtIO-MMIO devices for block storage and networking
- An initramfs that manages module loading, network setup, and rootfs pivoting
- Exclusive RAM and CPU pinning

4 VMM Engine: From KVM to Boot

The VMM engine (`vmm.rs`, ~1,170 lines) implements the full lifecycle of a micro-VM using direct KVM ioctls via the `rust-vmm` ecosystem—the same foundation powering AWS Firecracker and Intel Cloud Hypervisor.

4.1 KVM Initialization

1. Open `/dev/kvm`
2. `KVM_CREATE_VM` → VM file descriptor
3. `KVM_CREATE_IRQCHIP` → in-kernel PIC + IOAPIC
4. `KVM_CREATE_PIT2` → Programmable Interval Timer
5. Map guest memory → `GuestMemoryMmap` (two regions)
6. `KVM_CREATE_VCPU` → single vCPU (`id=0`)
7. Setup CPUID, SREGs, General Purpose Registers

4.2 Guest Memory Map (x86_64)

NKR uses a two-region memory model compatible with the Linux boot protocol:

Address	Content	Size
0x0000–0x9FFFF	Base RAM (conventional)	640 KB
0x0500	GDT (Global Descriptor Table)	32 bytes
0x7000	Zero Page (boot parameters)	4 KB
0x9000	PML4 (Page Map Level 4)	4 KB
0xA000	PDPTE (Page Directory Pointer)	4 KB
0xB000	PDE (Page Directory, 2 MB pages)	4 KB
0x20000	Kernel command line	variable
0x100000	bzImage load address	~10 MB
0x800_0000	Initramfs	variable

4.3 Boot Protocol

NKR implements the 32-bit Linux boot protocol for bzImage kernels:

1. **Kernel Load** — The bzImage is loaded at 0x100000 using `linux-loader::BzImage::load()`
2. **Initramfs Load** — Copied to 0x800_0000 in guest memory
3. **Zero Page Setup** — Boot parameters populated at 0x7000
4. **Page Table Write** — 2 MB pages with identity mapping via PML4 → PDPT → PD
5. **GDT Write** — A 4-entry table: null, code64, data, null
6. **vCPU Setup** — RIP points to the kernel entry point, RSI/RSP to the zero page

The kernel command line configures all VirtIO devices inline, and resolves critical clock synchronization issues by injecting specific clock parameters:

```
console=ttyS0 panic=1 pci=off noapic nolapic clocksource=jiffies tsc=nowatchdog
virtio_mmio.device=4K@0xd0000000:5      # network
virtio_mmio.device=4K@0xd0001000:6      # disk 0
virtio_mmio.device=4K@0xd0002000:7      # disk 1 (if exists)
root=/dev/vda rw init=/sbin/init nkr.ip=10.0.0.X
```

4.4 Time and Clock Management (Clock Synchronization)

Historically, micro-VMs experienced clock drift and hangs during boot due to the absence of a full hardware timer and the quirks of virtualized environments. NKR solves this issue holistically by implementing two key mechanisms:

1. **PIT2 (Programmable Interval Timer):** Explicitly instantiated (`KVM_CREATE_PIT2`) during VM initialization in KVM, providing the base system clock interrupt vital for the guest scheduler.
2. **Kernel Clock Sources:** Through the `clocksource=jiffies tsc=nowatchdog` parameter, the guest kernel is forced to rely on timed interrupts (jiffies) to advance time and the TSC (Time Stamp Counter) watchdog is disabled. This allows stable and reliable timekeeping even in high-density environments with extreme CPU contention where the TSC can present inconsistencies.

4.5 vCPU Loop

The core loop executes `KVM_RUN` continuously, dispatching four key exit reasons:

Exit Type	Handler
IoOut (I/O port write)	Serial console output (COM1 0x3F8)
IoIn (I/O port read)	Serial status registers
MmioWrite	Writes to VirtIO-MMIO registers (device config, queues, notifications)
MmioRead	Reads from VirtIO-MMIO registers (features, status, config space)
Hlt / Shutdown	Clean exit

`SIGTERM` is trapped with a signal handler that asserts an `AtomicBool`, triggering the vCPU loop to break and execute a clean shutdown sequence (unmounting volumes, dropping the TAP device, removing iptables rules).

5 VirtIO Device Model

NKR implements the VirtIO-MMIO (Memory-Mapped I/O) transport, not PCI, optimizing for extreme simplicity. The kernel boot parameter `pci=off` disables PCI enumer-

ation entirely.

5.1 MMIO Address Map

Address	Device	IRQ
0xD000_0000	VirtIO-Net (network)	5
0xD000_1000	VirtIO-Block disk 0 (rootfs)	6
0xD000_2000	VirtIO-Block disk 1	7
0xD000_3000+	Additional disks (+0x1000 each)	8+

5.2 VirtIO Block Device

Implementation: `block.rs` — 205 lines

- **Queue:** Single virtqueue, 256 descriptors
- **Sector size:** 512 bytes
- **Operations:** Read (type=0), Write (type=1)
- **Descriptor Chain:** Header (16 bytes: type + sector) → Data Buffer → Status Byte
- **Interrupts:** IRQ injected via `irqfd` after each batch is processed
- **Feature Negotiation:** `VIRTIO_F_VERSION_1` (bit 32)

The block device performs synchronous file I/O operations against the raw `.ext4` disk image—relying on the host kernel to provide robust page caching.

5.3 VirtIO Network Device

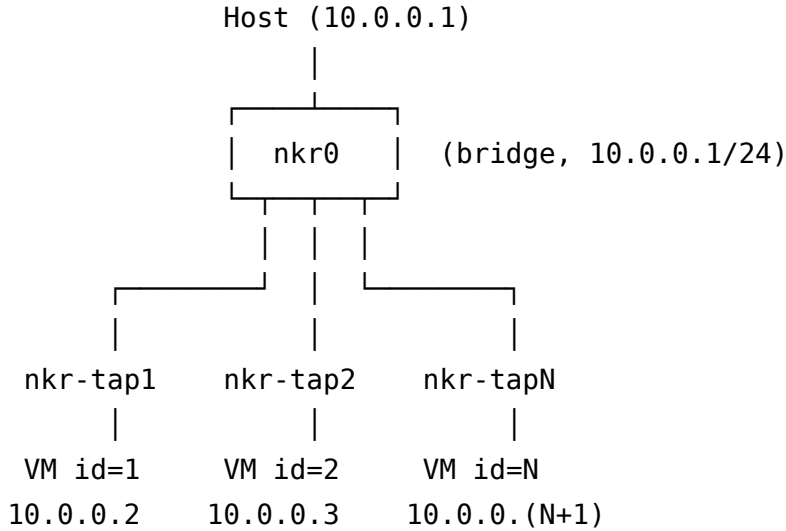
Implementation: `net.rs` — 286 lines

- **Queues:** Two virtqueues (RX=0, TX=1), 256 descriptors each
- **Backend:** Linux TAP device (`/dev/net/tun`, `TUNSETIFF`)
- **Header:** 12-byte VirtIO-net header (stripped/prepended on TX/RX)
- **RX Path:** A dedicated background thread executes blocking reads on the TAP fd, injects packets into the RX queue, and signals the IRQ
- **TX Path:** Upon QueueNotify (queue 1), it reads descriptor chains from the TX queue, strips the header, and writes to the TAP interface
- **Features:** `VIRTIO_NET_F_MAC` | `VIRTIO_NET_F_STATUS` | `VIRTIO_F_VERSION_1`
- **Config Space:** 6-byte MAC address + 2-byte link status

6 Networking

6.1 Bridge Topology

NKR creates and manages a Linux bridge `nkr0` using the `10.0.0.0/24` subnet:



6.2 Automatic Configuration

For each VM, the VMM:

1. Creates the TAP device `nkr-tap{vm_id}`
2. Attaches it to the `nkr0` bridge
3. Assigns the MAC `52:54:00:12:34:{vm_id}`
4. Passes the IP `10.0.0.{vm_id + 1}` to the guest via kernel cmdline (`nkr.ip=`)
5. Configures iptables rules:
 - POSTROUTING MASQUERADE for internet access
 - FORWARD ACCEPT for inter-VM traffic
 - PREROUTING DNAT + OUTPUT DNAT for port forwarding

6.3 Port Forwarding

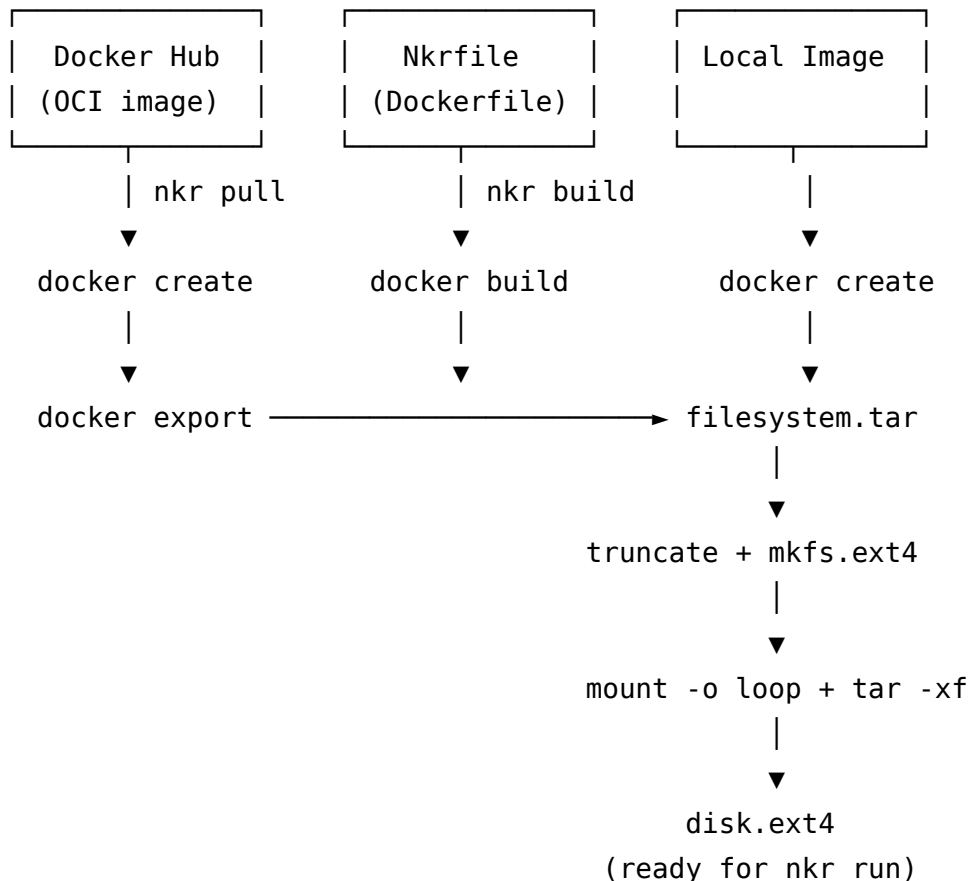
```
nkr run --disk odoo.ext4 --port 8069:8069 --id 2
# Creates: host:8069 → 10.0.0.3:8069 (DNAT + MASQUERADE)
```

Rules are automatically torn down when the VM shuts down via `cleanup_port_forwarding()`.

7 Disk Lifecycle: From OCI to ext4

NKR uses Docker exclusively as a **build tool** to transform OCI images into raw ext4 filesystems. Docker is completely removed from the runtime equation.

7.1 Build Pipeline



7.2 Nkrfile Format

Nkrfiles are standard Dockerfiles. NKR provides templates for common services:

```

# Nkrfile.pg – PostgreSQL 15
FROM postgres:15
ENV POSTGRES_USER=odoo
ENV POSTGRES_PASSWORD=odoo

# Nkrfile.odoo – Odoo 17
FROM odoo:17.0
USER root
COPY deploy/config/odoo.conf /etc/odoo/odoo.conf

```

```
RUN mkdir -p /mnt/extra-addons && chown odoo:odoo /mnt/extra-addons  
USER odoo
```

7.3 Copy-on-Write Snapshots

For multi-tenant deployments, NKR creates CoW snapshots from a base disk:

```
cp --reflink=auto odoo-base.ext4 client1.ext4
```

On filesystems supporting reflinks (btrfs, XFS with reflink), this operation is instantaneous and consumes zero additional disk space until writes diverge. On other filesystems, NKR falls back to `cp --sparse=always`.

7.4 Volumes

NKR provides a volume system to inject configuration and persist data:

- **Pre-boot injection:** The root disk is loop-mounted and files are copied from host to guest paths
- **Post-shutdown extraction:** Volumes marked with `:rw` are copied back from guest to host
- **Syntax:** `host_path:guest_path (read-only)` or `host_path:guest_path:rw`

```
nkr run --disk odoo.ext4 \  
  --volume ./odoo.conf:/etc/odoo/odoo.conf \  
  --volume /opt/data/filestore:/var/lib/odoo:rw
```

7.5 Environment Variables

Environment variables are written to `/etc/nkr-env` inside the root disk prior to boot:

```
nkr run --disk pg.ext4 --env POSTGRES_USER=odoo --env POSTGRES_PASSWORD=secret
```

The initramfs sources this file during early boot, making the variables available to the guest init process.

8 The CPU Model: “Chrs”

NKR introduces a CPU allocation unit dubbed the **chr** (pronounced “core”):

Value	Meaning
1 chr	20% of a physical core
5 chrs	1 full physical core
10 chrs	2 physical cores

8.1 Implementation

CPU allocation is enforced using `sched_setaffinity()`:

```
let cores_needed = ((chrs as f32) / 5.0).ceil() as u32;
let cores_to_use = cores_needed.min(num_cpus);
// Pin the vCPU thread to cores [0..cores_to_use]
sched_setaffinity(0, &cpuset);
```

Chrs are **exclusive**—the VM process is pinned to dedicated physical cores, preventing contention with other VMs.

Deployment Example (8-core server):

Service	Chrs	Cores Used
PostgreSQL	10 (2 cores)	Cores 0-1
Odoo #1	5 (1 core)	Core 2
Odoo #2	5 (1 core)	Core 3
Odoo #3-#8	1 each	Cores 4-7 (shared pool)

9 Initramfs Generation

NKR bundles an automatic initramfs generator (`initramfs.rs`, 383 lines) that crafts tailored boot environments for each service.

9.1 Boot Sequence

Initramfs boots (PID 1)

```
|
├─ Mount /proc, /sys, /dev
```

```

└─ Load kernel modules:
|   crc32c → libcrc32c → crc16 → mbcache → jbd2 → ext4
|   virtio_blk → failover → net_failover → virtio_net
|
└─ Wait for /dev/vda (up to 3 seconds)
└─ Parse nkr.ip= from /proc/cmdline
└─ Configure eth0 with IP/24, gateway → 10.0.0.1
|
└─ Mount /dev/vda → /newroot (ext4)
└─ Mount extra disks /dev/vdb..vde → /newroot/mnt/disk0..3
└─ Bind-mount /proc, /sys, /dev into /newroot
|
└─ Write /etc/nkr-net.sh (network config script)
└─ Write /etc/resolv.conf (DNS: 8.8.8.8, 8.8.4.4)
└─ Setup networking via chroot
|
└─ Detect init: /sbin/init → systemd → Docker entrypoint
└─ Build wrapper /sbin/nkr-init:
|   - Source /etc/nkr-env (NKR environment variables)
|   - Execute the detected init
|
└─ exec switch_root /newroot /sbin/nkr-init

```

9.2 Automatic Entrypoint Detection

When building via `nkr pull` or `nkr build`, NKR:

1. Extracts `ENTRYPOINT` + `CMD` from the Docker image metadata
2. Mounts the disk read-only and scans for known entrypoint scripts (`/entrypoint.sh`, `/docker-entrypoint.sh`, etc.)
3. Generates a bespoke init script that loads NKR variables and launches the correct entrypoint

This enables NKR to boot unmodified Docker images—PostgreSQL, nginx, Redis, Odoo—as micro-VMs without any image tampering.

10 Orchestration with Compose

NKR features a compose system (`compose.rs`, 840 lines) modeled after Docker Compose but engineered for VM orchestration.

10.1 Compose File Format

```
services:
  db:
    disks: [/opt/nkr/disks/postgres.ext4]
    ram: 512
    chrs: 1
    ports: ["5432:5432"]
    volumes: ["/opt/nkr/data/pg:/var/lib/postgresql/data:rw"]
    healthcheck:
      port: 5432
      initial_delay: 15
      interval: 5
      retries: 12

  odoo:
    disks: [/opt/nkr/disks/odoo-prod.ext4]
    build:
      nkrfile: Nkrfile.odoo
      size_mb: 4096
    ram: 1024
    chrs: 2
    ports: ["8069:8069"]
    volumes:
      - "/opt/nkr/config/odoo.conf:/etc/odoo/odoo.conf"
      - "/opt/nkr/modules:/mnt/extra-addons"
    environment:
      DB_HOST: "10.0.0.2"
```

10.2 Resource Resolution

NKR compose resolves resources intelligently, walking a priority chain:

Resource	Resolution Order
Disk	YAML path → <yaml_dir>/<name> → /mnt/nkr/images/<name>
Kernel	Explicit → <yaml_dir>/bzImage → /mnt/nkr/kernel/bzImage → alongside nkr binary
Initramfs	Explicit → by service name → by disk name → heuristic → auto-generation

10.3 Features

- **Auto-build:** Automatically builds the disk if a service defines a `build:` section and the disk is absent
- **Health checks:** TCP monitoring with configurable delay, interval, and retries
- **Daemon mode:** `nkr compose up -d` executes in the background with log rotation (max 10 MB, 3 rotated files)
- **CoW Snapshots:** Automatically crafts snapshots when a base disk is already locked by another VM
- **Deterministic IDs:** Services are alphanumerically sorted; IDs are deterministically granted using a persistent registry (`/mnt/nkr/registry.json`)

10.4 NKR Data Directory

```

/mnt/nkr/                                # Default (NKR_DATA_DIR variable)
├─ images/                               # Base ext4 disk images
├─ initramfs/                             # .cpio.gz files per service
│   ├─ base/                             # busybox + kernel modules (shared)
│   └─ pg.cpio.gz
│   └─ odoo.cpio.gz
├─ kernel/                               # Shared bzImage
├─ snapshots/                             # CoW snapshots per stack
└─ registry.json                         # Persistent name → ID map

```

11 Multi-Tenant Deployment

NKR ships with an end-to-end deployment toolkit tailored for multi-tenant Odoo 17.

11.1 Tenant Registry

Clients are defined in `deploy/clients.yml`:

```
global:
  pg_ram: 2048
  odoo_ram: 256
  odoo_chrs: 1
  base_disk: /opt/nkr/disks/odoo-base.ext4

clients:
  - name: acme
    domain: acme.example.com
    db_name: acme_prod
  - name: globex
    domain: globex.example.com
    db_name: globex_prod
    ram: 512          # override
    chrs: 2           # override
```

11.2 Provisioning Pipeline

`mt-provision.sh <client-name>`

```
|
|─ Create CoW disk:      cp --reflink=auto base.ext4 → <client>.ext4
|─ Craft Odoo config:    odoo.conf with db_name, admin_passwd, workers=0
|─ Craft nginx config:   <domain> → 10.0.0.<vm_ip>:8069
|─ Link nginx site:      ln -s sites-available → sites-enabled
|─ Reload nginx:         nginx -s reload
```

11.3 Hot Module Update

The `deploy/update.sh` script facilitates near-zero downtime module updates:

Mode	Command	Downtime
Production	<code>update.sh</code>	~5 seconds
Test	<code>update.sh --test</code>	0 (runs on port 8070)
Rollback	<code>update.sh --rollback</code>	~5 seconds

Mode	Command	Downtime
DB Update	<code>update.sh --update-db</code>	~30 seconds

Update Flow:

1. Automatically back up current modules (retains the last 5)
2. Stop the Odoo VM (PostgreSQL remains running)
3. Mount the disk, rsync modules featuring `__manifest__.py`
4. Start the Odoo VM → ~5s total downtime

11.4 Target Architecture

Server (16–32 GB RAM)

```

|
├─ 1× PostgreSQL VM (1–2 GB RAM, 2 chrs)
|   └─ Holds every DB for all 50 clients
|
├─ 50× Odoo VMs (~256 MB each, 1 chr)
|   ├── acme      (id=2, 10.0.0.3:8069)
|   ├── globex    (id=3, 10.0.0.4:8069)
|   └─ ...
|
├─ nginx (on the host, not in a VM)
|   └─ Reverse proxy + SSL per domain
|
└─ Exposed ports: 80, 443, 5566 (SSH)
    Everything else is strictly internal on nkr0

```

Resource Budget for 50 Tenants:

Component	RAM	Disk
PostgreSQL	2 GB	2 GB (shared)
50× Odoo	$50 \times 256 \text{ MB} = \mathbf{12.5 \text{ GB}}$	~4 GB base + CoW deltas
NKR Overhead	$\sim 50 \times 5 \text{ MB} \approx 250 \text{ MB}$	~3 MB binary
Total	~15 GB	~10 GB (vs ~75 GB Docker)

12 Observability and Metrics

NKR incorporates a low-level telemetry system built directly into the hypervisor that measures and exposes real-time resources utilized by each micro-VM, bypassing the need to deploy additional agents within the guests.

The metrics engine extracts intelligence via lightweight probes against `procfs` and the host networking subsystem:

- **CPU%:** Calculated via a synchronous 200 ms sampling window analyzing `/proc/{pid}/stat`. Engineered to cushion computational drag, the polling interval is shared globally if inspecting multiple simultaneous VMs.
- **RAM (VmRSS):** NKR audits the actual physical memory (RSS) consumed on the server by parsing `/proc/{pid}/status`. This enables an accurate, zero-overhead visualization of megabytes freed or constrained against the VM's pre-allocated RAM.
- **Disk (I/O):** Accumulated read and write bytes traversing the root block and database volumes (`/proc/{pid}/io`).
- **Network (TAP):** Volumetric transmit and receive counters on the emulated network interface (TAP) to meter cross-border bandwidth between the guest and the exterior using `/proc/net/dev`.
- **KSM State (Kernel Same-page Merging):** Instantaneous oversight of Linux's memory page deduplicator. NKR's CLI leverages this to compute "Megabytes Saved" by quantifying in real-time the proportion of identical memory pages shared amongst Odoo micro-VMs—critical for sustaining extreme hyper-density.

All the aforementioned data can be surveyed in the console via a tabular tenant-by-tenant layout using:

```
sudo nkr ps --stats
```

This unmediated telemetry affords operators total, instantaneous visibility into the infrastructural tax exacted by each client with near zero host impact.

13 Comparison with Existing Solutions

13.1 NKR vs Docker

Dimension	Docker	NKR
Isolation	Shared kernel (namespaces + cgroups)	Full VM (KVM, separate kernel)
Kernel Vulnerability	Impacts all containers	Impacts only the guest VM
CPU Guarantee	cgroups shares (soft limit)	Core pinning (strict limit)
RAM	Overcommit by default	Exclusive, no overcommit
Binary Size	dockerd ~100 MB + containerd + runc	~2-4 MB single binary
Boot Time	~1-3s (process initialization)	~1-2s (complete VM boot)
Disk Format	Layered overlay filesystem	Raw ext4 (CoW snapshots)
Networking	veth + bridge	TAP + bridge + iptables
Compose	docker-compose (YAML)	nkr compose (YAML, compatible syntax)

13.2 NKR vs Firecracker

Dimension	Firecracker	NKR
Language	Rust	Rust
KVM Interface	Direct (kvm-ioctls)	Direct (kvm-ioctls)
VirtIO	MMIO	MMIO
Focus area	Serverless (AWS Lambda)	Multi-tenant SaaS (Odoo)
Disk plumbing	External	Built-in (nkr pull/build, OCI→ext4)
Orchestration	None (external: containerd)	Built-in (nkr compose)
MT Tooling	None	Comprehensive (clients.yml, provisioning)
Volume Injection	External	Built-in (pre-boot loop mount)
CPU Model	Standard vCPU	“Chrs” (20% granularity + pinning)

Dimension	Firecracker	NKR
Code Lines	~70,000+	~4,000 (tightly focused scope)

13.3 NKR vs QEMU/KVM

Dimension	QEMU/KVM	NKR
Binary Size	~20-50 MB	2-4 MB
Device Model	Heavyweight x86 prep (PCI, USB, ACPI...)	Minimal VirtIO-MMIO only
Configuration	Complex CLI / libvirt XML	Straightforward CLI flags / YAML
Boot Time	~3-10 seconds	~1-2 seconds
Dependencies	libvirt, qemu, virt-manager	None (just /dev/kvm)
Attack Surface	Large (full emulation)	Minimal (3 MMIO devices)

14 Security Model

14.1 Isolation Boundaries

Layer	Mechanism
CPU	KVM hardware virtualization (VT-x/AMD-V). Guest runs in ring 0 of an isolated address space.
Memory	GuestMemoryMmap creates dedicated memory regions. No shared memory between VMs.
Disk	Each VM owns a disparate ext4 file. No shared overlay filesystem.

Layer	Mechanism
Network	Dedicated TAP device per VM. Per-VM iptables isolation rules.
Process	Every VM runs as a discrete host process tied to its own PID. SIGTERM/SIGKILL for lifecycles.

14.2 Attack Surface

The attack surface of NKR is fundamentally smaller than both Docker and QEMU:

- **No Userspace Device Emulation** (vs QEMU): Only 3 lean MMIO device handlers
- **No Shared Kernel** (vs Docker): A kernel exploit within the guest cannot pivot to the host
- **No Container Escape Routes**: No namespaces, no cgroups, no procfs cross-talk
- **Minimal Host Interaction**: I/O is restricted entirely to file access (disk), TAP read/writes (network), and serial dumping

14.3 Operational Security

- Externally, only ports 80, 443, and SSH (configurable) are exposed
- All inter-VM broadcast traffic is effectively confined to the nkr0 bridge (10.0.0.0/24)
- Mandates root access for KVM/TAP/iptables configuration (by design—no rootless mode)

15 Limitations and Future Work

15.1 Current Limitations

Limitation	Impact	Planned Resolution
Single vCPU per VM	Nullifies SMP inside guests	Multi-vCPU support (medium priority)

Limitation	Impact	Planned Resolution
VirtIO-MMIO only	Zero PCI passthrough capabilities	Adequate for target SaaS workloads
No VirtIO-FS	Volumes require pre/post-boot copying	VirtIO-FS for live directory sharing
No Live Migration	VM must be halted to migrate between hosts	Future consideration
No Hot Snapshots	VM must be halted to snapshot the disk	Future consideration
No Automated Testing	Manual verification exclusively	CI test suite integration
Linux Host Only	Mandates a Linux environment with KVM	Intended by design

15.2 Roadmap

High Priority:

- Generate `nkr-compose.yml` for multi-tenant stacks automatically (`mt-compose-gen.sh`)
- End-to-end validation deploying N authentic clients
- Automated Let's Encrypt tooling via `certbot`

Medium Priority:

- Multi-vCPU enablement
- VirtIO-FS for hyper-efficient volume sharing
- Real-time resource dashboarding (RAM/CPU/disk) per VM
- Automated per-tenant PostgreSQL snapshotting

Low Priority:

- Live VM migration across bare-metal clusters
- Live disk snapshots without VM interruption
- Web-based fleet management panel

16 Conclusion

NKR demonstrates unequivocally that attaining **container-level density and operational simplicity** paired with **VM-level isolation and strict resource framing** is possible, requiring fewer than 4,000 lines of Rust packed into a 2-4 MB statically linked binary sporting zero runtime dependencies.

For service operators sustaining dozens of SaaS tenants strapped to a single commercial server, NKR constitutes a fundamentally different tradeoff compared to Docker or classical Virtual Machines:

- **Every tenant receives unquestionable hardware isolation**, not merely namespace separation
- **Every tenant enjoys guaranteed throughput**, evading chaotic shared memory pools
- **Operators preserve their Docker workflow**, leveraging familiar build, run, and compose semantics
- **The infrastructure topology consolidates natively**: 1 PostgreSQL backing N Odoo processes and 1 nginx, instead of N complete overlapping stacks

NKR is opinionated software. Instead of trying to be a sprawling general-purpose hypervisor like QEMU or an unopinionated container nexus like Kubernetes, NKR narrows its sights onto a high-value operational payload: **Hyper-dense, multi-tenant bare-metal SaaS**. This focused scope permits the hypervisor to be simple enough to comprehend entirely, slight enough to audit exhaustively, and brisk enough to boot in seconds.

17 Appendix A: Technology Stack

Component	Technology	Version
Language	Rust	Edition 2021
KVM Interface	kvm-ioctls	0.19
KVM Bindings	kvm-bindings	0.10
Guest Memory	vm-memory (GuestMemoryMmap)	0.14
Kernel Loader	linux-loader (bzImage)	0.11
VirtIO Queues	virtio-queue	0.12
CLI Argument Parsing	clap (derive)	4.x
Serialization	serde + serde_yaml + serde_json	1.x / 0.9 / 1.x
System Utilities	vmm-sys-util	0.12
Guest Kernel	Linux bzImage	6.6.117-0-virt

18 Appendix B: Source Code Metrics

Module	File	Lines	Responsibility
VMM Engine	vmm.rs	1,171	KVM init, PIT2 clock, boot protocol, MMIO dispatch
Compose	compose.rs	839	YAML parsing, orchestration, health checks, daemon mode
Initramfs	initramfs.rs	382	Auto-generation of bespoke boot environments
Metrics	metrics.rs	331	Telemetry, /proc analytics, RSS status, I/O and KSM

Module	File	Lines	Responsibility
Networking	net.rs	285	VirtIO-net device, TAP backend, RX/TX threading
State	state.rs	252	VM registry, lifecycle supervision, nkr ps
Block	block.rs	204	VirtIO-block device, raw sector I/O
Pull	pull.rs	201	Docker Hub → ext4 pipeline
Build	build.rs	192	Nkrfile → ext4 pipeline
Registry	registry.rs	166	Persistent name-to-ID mapping
CLI	cli.rs	159	Command line interface specifications
Main	main.rs	148	Entry point, command routing
Total		3,999	

19 Appendix C: Quick Start

```
# Compile NKR from source
```

```
cargo build --release
```

```
# Binary yields: target/release/nkr (~2–4 MB)
```

```
# Pull a PostgreSQL image and synthesize an ext4 disk
```

```
sudo ./target/release/nkr pull postgres:15 postgres.ext4 --size-mb 2048
```

```
# Spin up a micro-VM
```

```
sudo ./target/release/nkr run \  
  --disk postgres.ext4 \  
  --ram 512 \  
  --chrs 1 \  
  --id 1 \  
  --port 5432:5432
```

```
# Verify running VMs
```

```
sudo ./target/release/nkr ps
```

```
# Halt a VM
```

```
sudo ./target/release/nkr stop 1
```

```
# Orchestrate a multi-service stack autonomously
```

```
sudo ./target/release/nkr compose up -f nkr-compose.yml -d
```

NKR is open-source software. Contributions and feedback are highly appreciated.

© 2026 NKR Contributors. MIT Licensed.